A Toy Paper for `pycasso`

An Example Author

Contents

Abstract

This is a synthetic paper, generated only to demonstrate `pycasso`'s project-wide restyling on a document with several figures, each plotted with a different, mismatched colour scheme – the kind of inconsistency that accumulates naturally across a real paper's revision history, as different co-authors add figures with whatever colour scheme their own plotting habits default to. None of the numbers below mean anything; only the colours, and how `pycasso` reassigns them, do. Every figure's caption names exactly which colour source it used, linked, so you can look it up and compare it against whatever `pycasso` decided to do with it.

1 Introduction

`pycasso` recolours the figures of a paper – and, when given the project's own LaTeX source rather than just the compiled PDF, its hyperlink colours and code-listing syntax highlighting too – onto a consistent palette (see https://github.com/MatteoRobbiati/pycasso). It decides how many independent colours a document really uses by clustering hues, then assigns each one a distinct replacement from the target palette whenever the palette has enough hue families to offer one. Sections 2 and 3 below run through the two kinds of figure it treats differently: a handful of discrete series colours, versus a genuinely continuous colormap encoding a continuous quantity.

2 Discrete, categorical figures

The four figures in this section each use a small number of distinct, named colours – exactly the case `pycasso.fit()` is built around. Figure 1 uses matplotlib's default qualitative cycle, tab10. Figure 2 uses seaborn's Set2. Figure 3 uses matplotlib's Dark2. Figure 4 uses matplotlib's Accent. None of these four sources were designed with one another in mind, which is exactly the point: a real paper accretes exactly this kind of mismatch one figure at a time.

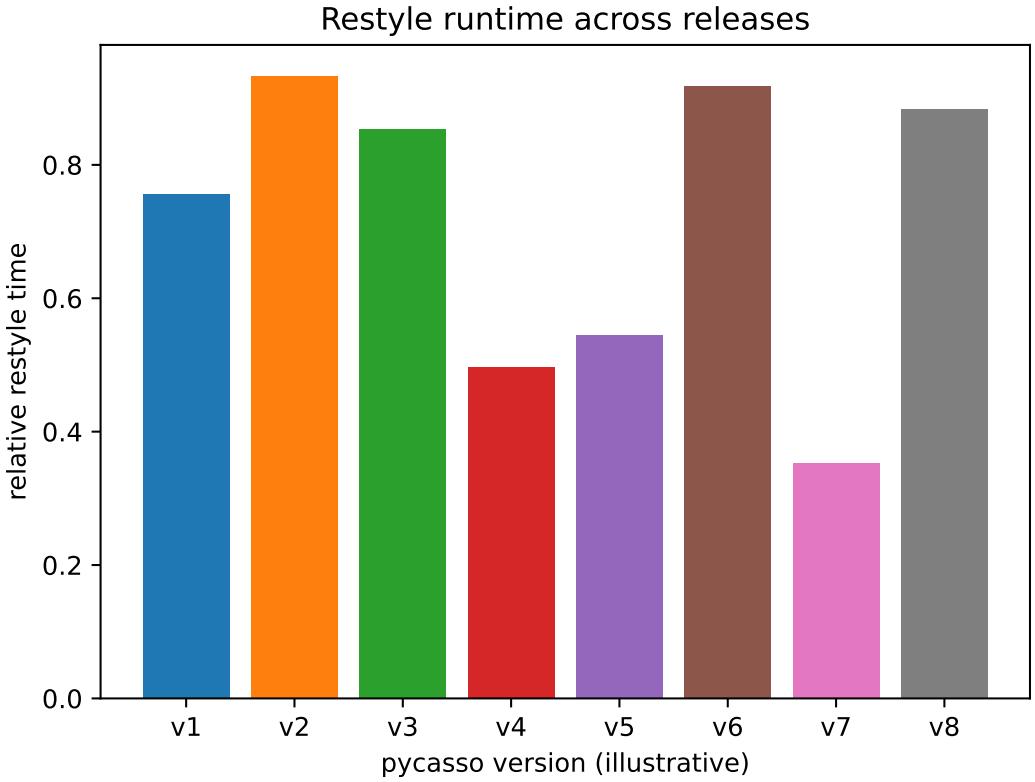

Figure 1: Restyle runtime across releases (illustrative numbers). Colour source: matplotlib tab10.

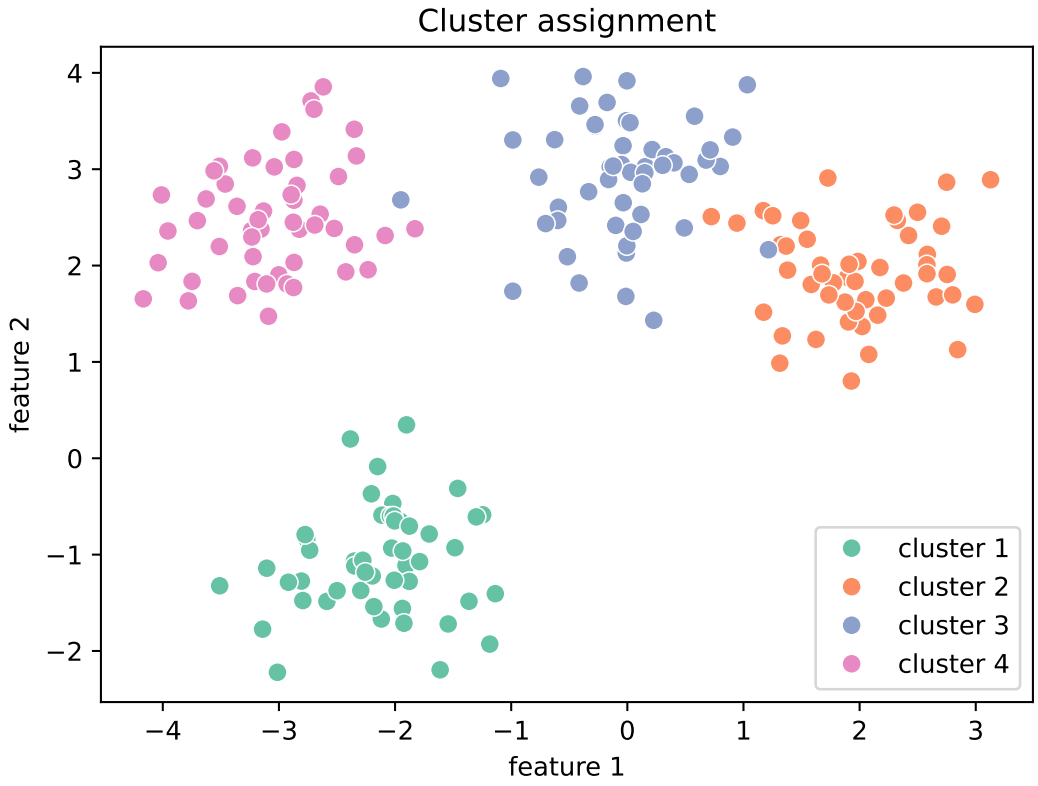

Figure 2: Cluster assignment in a toy feature space. Colour source: seaborn Set2.

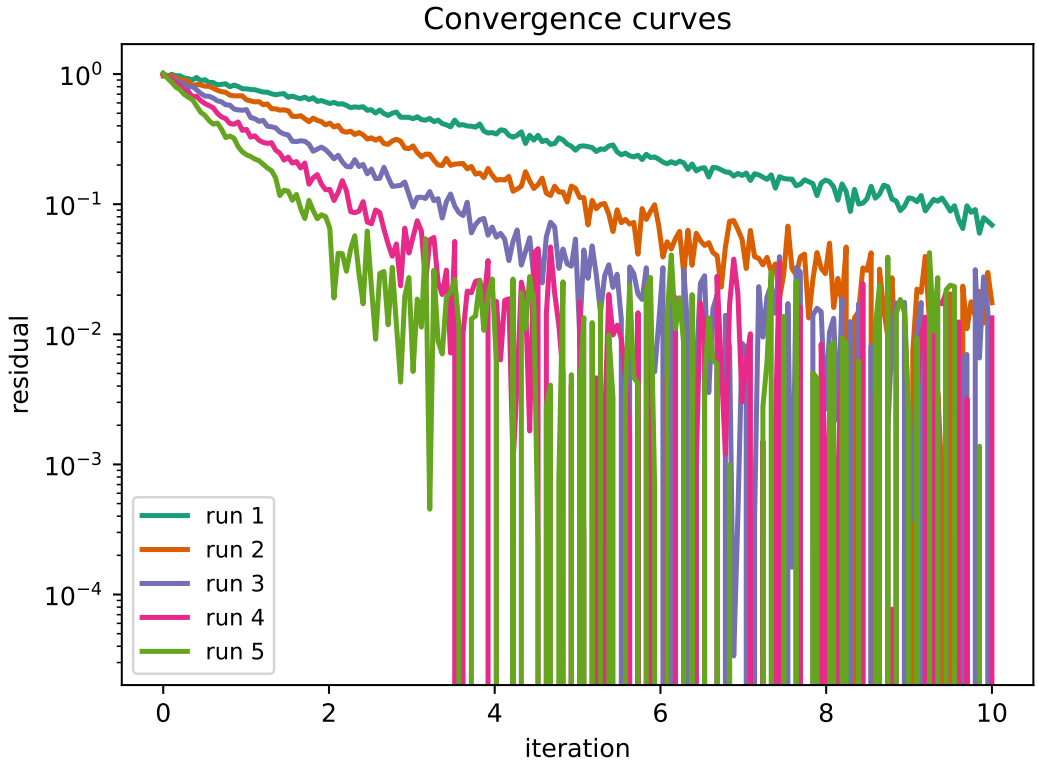

Figure 3: Convergence curves for five synthetic runs. Colour source: matplotlib Dark2.

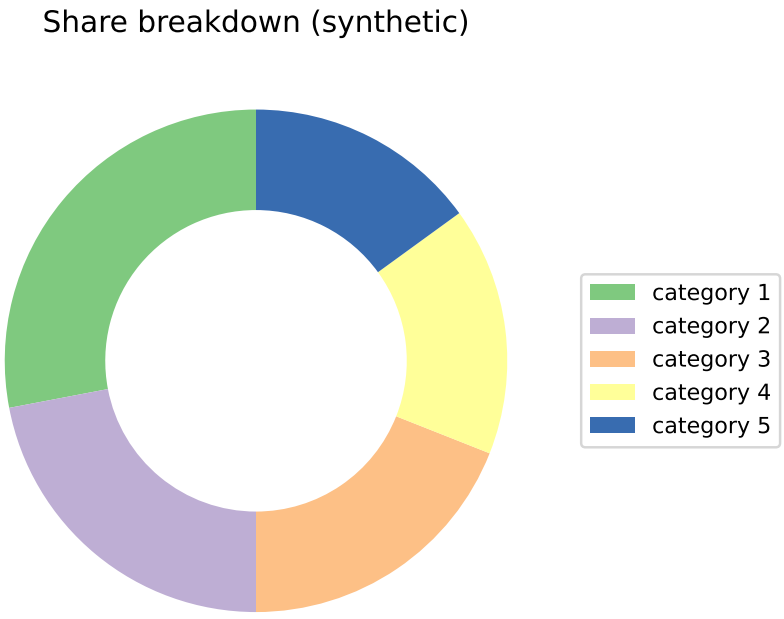

Figure 4: A synthetic share breakdown. Colour source: matplotlib Accent.

3 Continuous colormaps

Not every figure uses a handful of discrete colours. The three figures below use unrelated *continuous* colormaps: a diverging one, and two sequential ones with entirely different hues. `pycasso` recognises this (see `pycasso.cluster.looks_continuous`) and switches to `pycasso.fit_colormap`, which finds each figure's own one or two dominant hues and blends continuously between their replacements, instead of snapping a smooth gradient onto a handful of discrete colours and introducing visible banding.

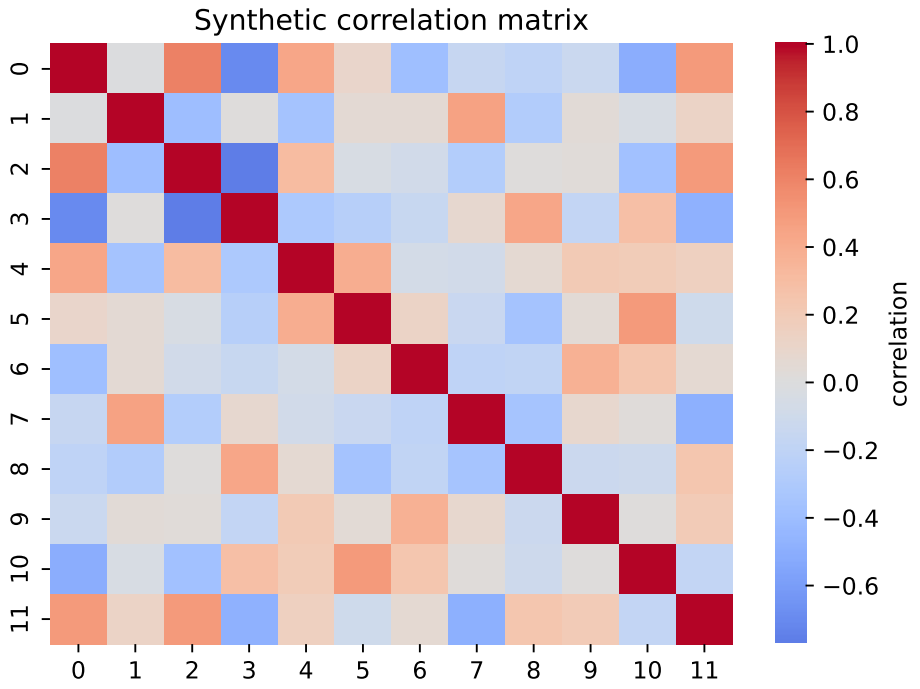

Figure 5: A synthetic correlation matrix. Colour source: matplotlib/seaborn coolwarm (diverging).

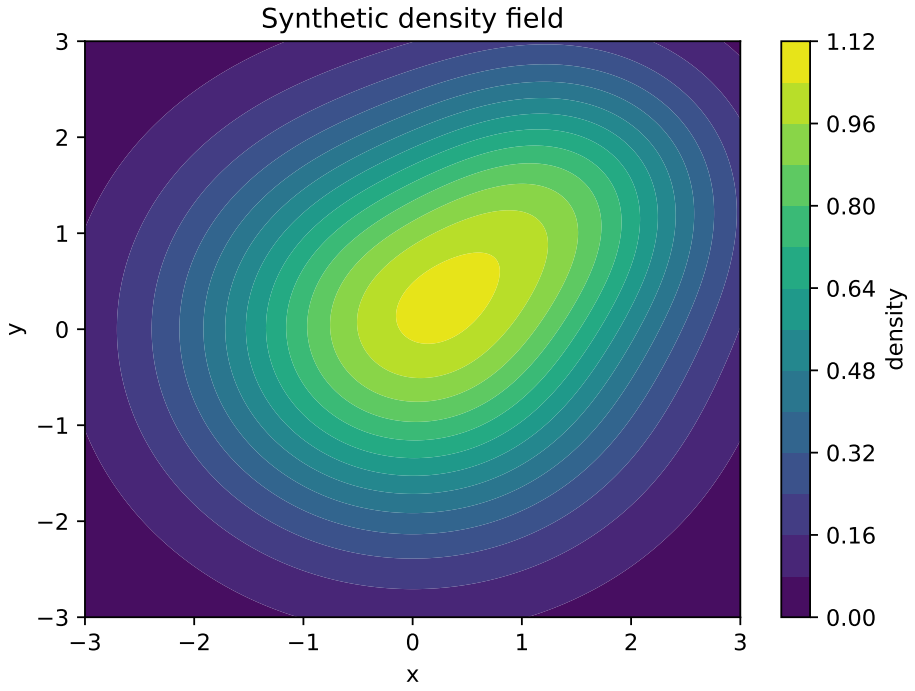

Figure 6: A synthetic density field. Colour source: matplotlib viridis (sequential).

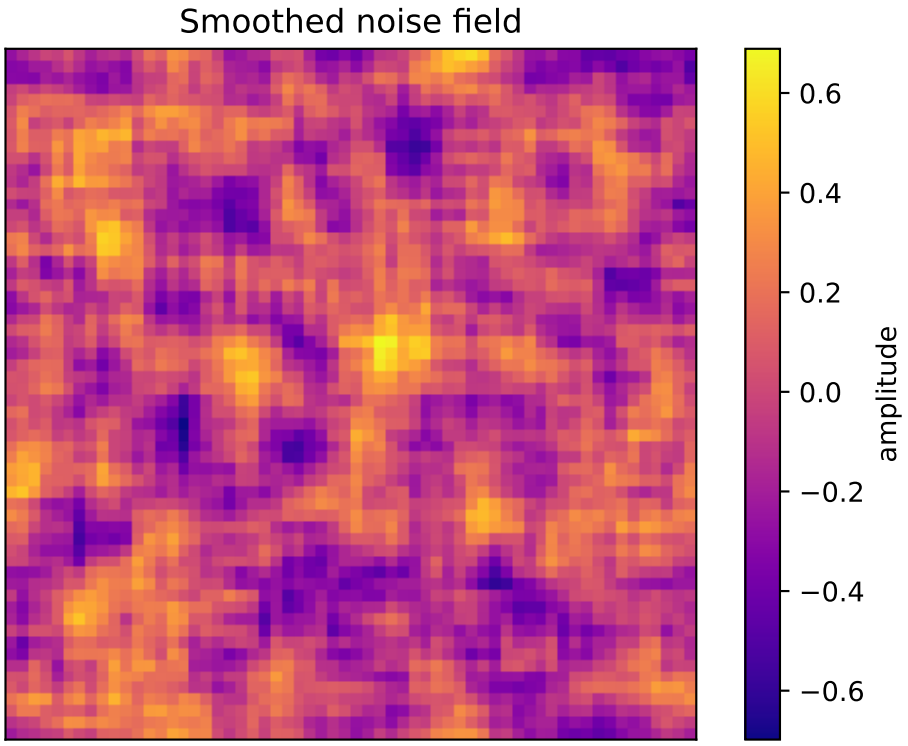

Figure 7: A smoothed noise field. Colour source: matplotlib plasma (sequential, unrelated hue to viridis).

4 A raster logo

`pycasso` works on plain images too, not just figures inside a paper. Figure 8 is a flat PNG, included here to exercise that same path from inside a full project restyle. Its own colours (a particular yellow and red) happen to already sit close to some palettes' own yellow and red, so a restyle onto one of those can look almost unchanged at a glance – that is `mode="hue"` correctly doing very little when there is very little to do, not a sign that nothing happened; a palette with unrelated hues (e.g. `violet`) changes it clearly.

Figure 8: A raster logo, recoloured alongside everything else. Original colours: flat yellow and red, no particular named source.

5 Using pycasso

Restyling the compiled PDF alone reaches only what ended up embedded as a drawing object:

```
import pycasso

palette = pycasso.Palette.load("okabe-ito")
paper = pycasso.Painter("main.pdf")
fitted = pycasso.fit(paper, palette)
paper.restyle(fitted, mode="hue")
paper.save("main_restyled.pdf")
```

Working from the project's own source, as this paper does, also reaches hyperlink colours and code-listing syntax highlighting:

```
import pycasso

palette = pycasso.Palette.load("persian")
project = pycasso.Project("toy_paper.zip")
fitted = pycasso.fit(project, palette)
project.restyle(fitted, mode="hue")
project.save("toy_paper_restyled.zip")
```

See https://github.com/MatteoRobbiati/pycasso for the full API.

6 Conclusion

This document exists only to be recoloured.